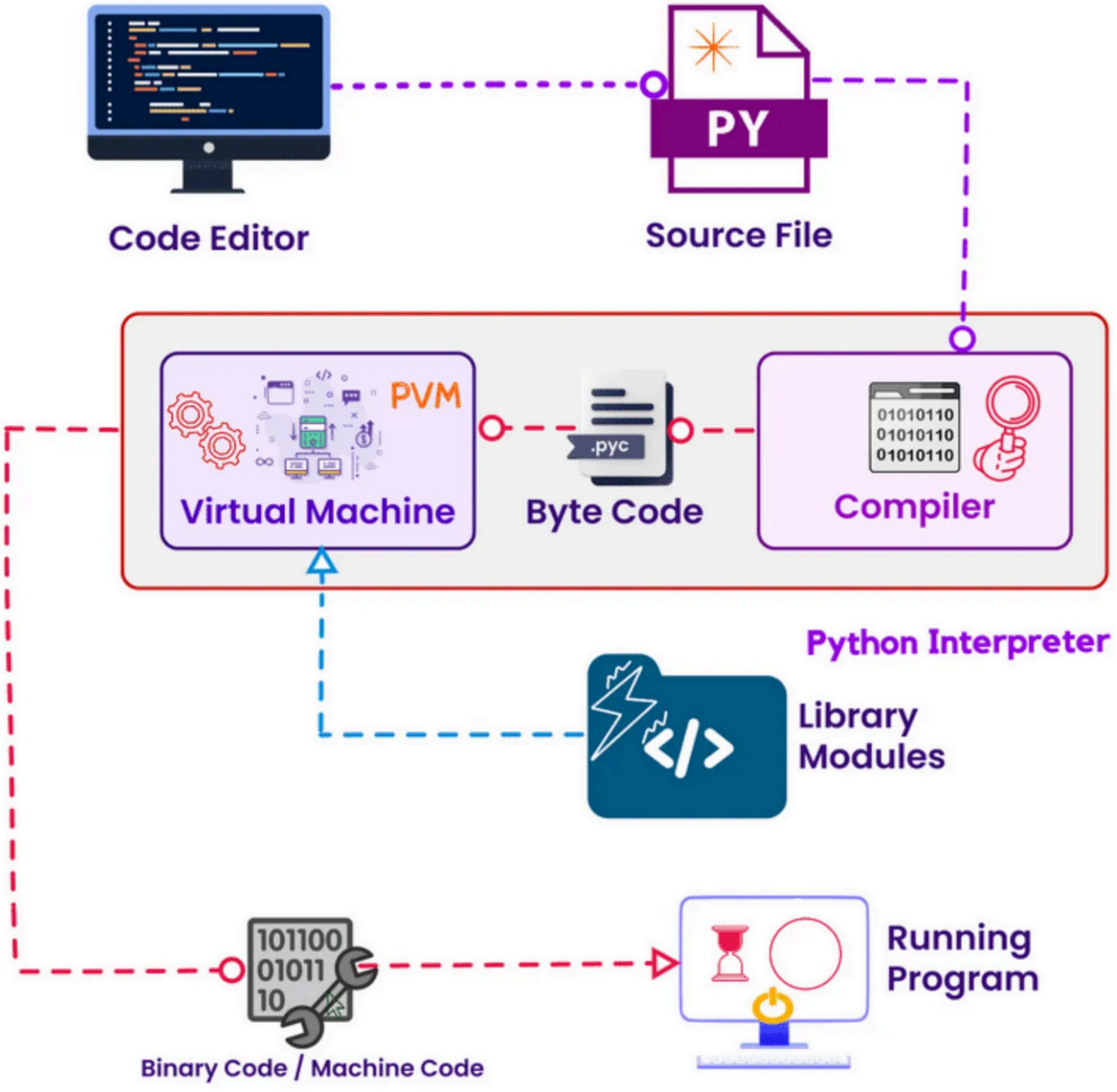
MDL Tutorial 1

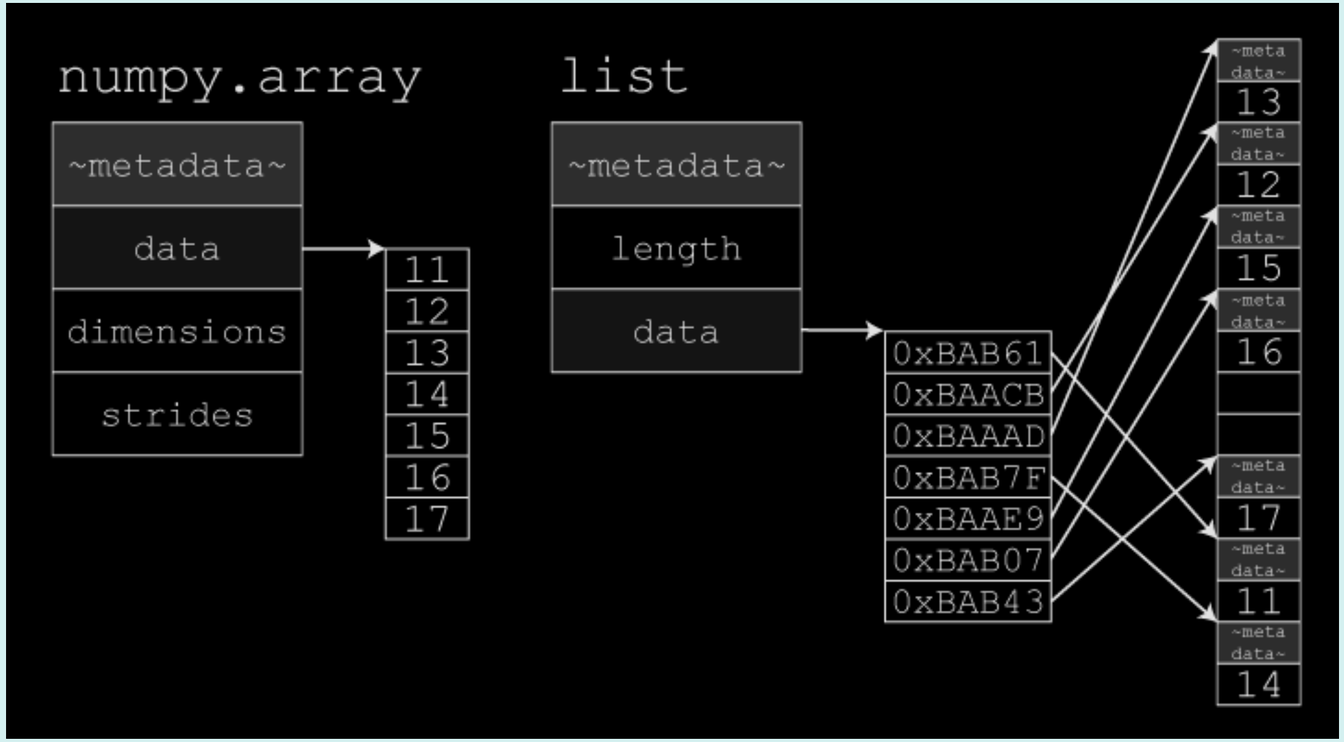
Presented by Medha and Tejasvini

Python- Dynamic Interpreter



Python- Numpy

Python is easy to write but not very fast, because it runs through a virtual machine. Now the natural question is: if Python is slow, how do we do fast math and machine learning in Python?



Almost every machine learning library is built on top of NumPy. In a NumPy array, all elements are of the same type - for example, all integers or all floats.

When we write NumPy code in Python, the heavy computation actually happens in C. NumPy stores data in one continuous block of memory. This idea of keeping data close together in memory is called spatial locality. Because NumPy arrays use spatial locality, they are much faster than Python lists.

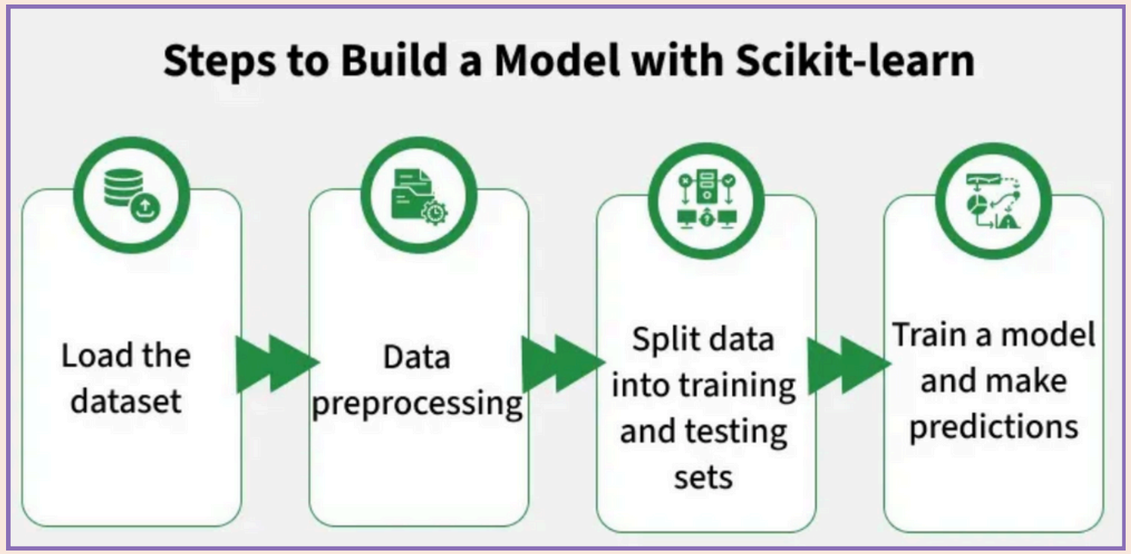
What is sci-kit learn

scikit-learn is a **machine learning library** built on top of NumPy.

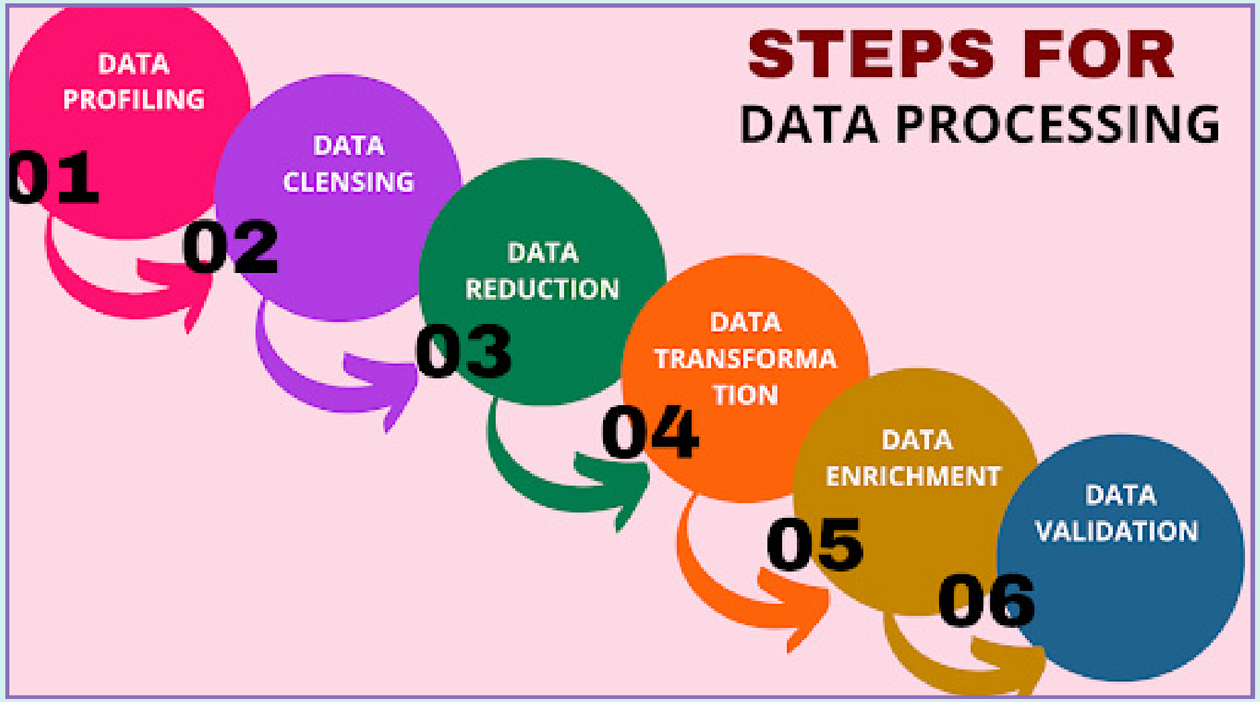
It gives us **ready-to-use tools** for **data preprocessing, machine learning models,** and **evaluation**.

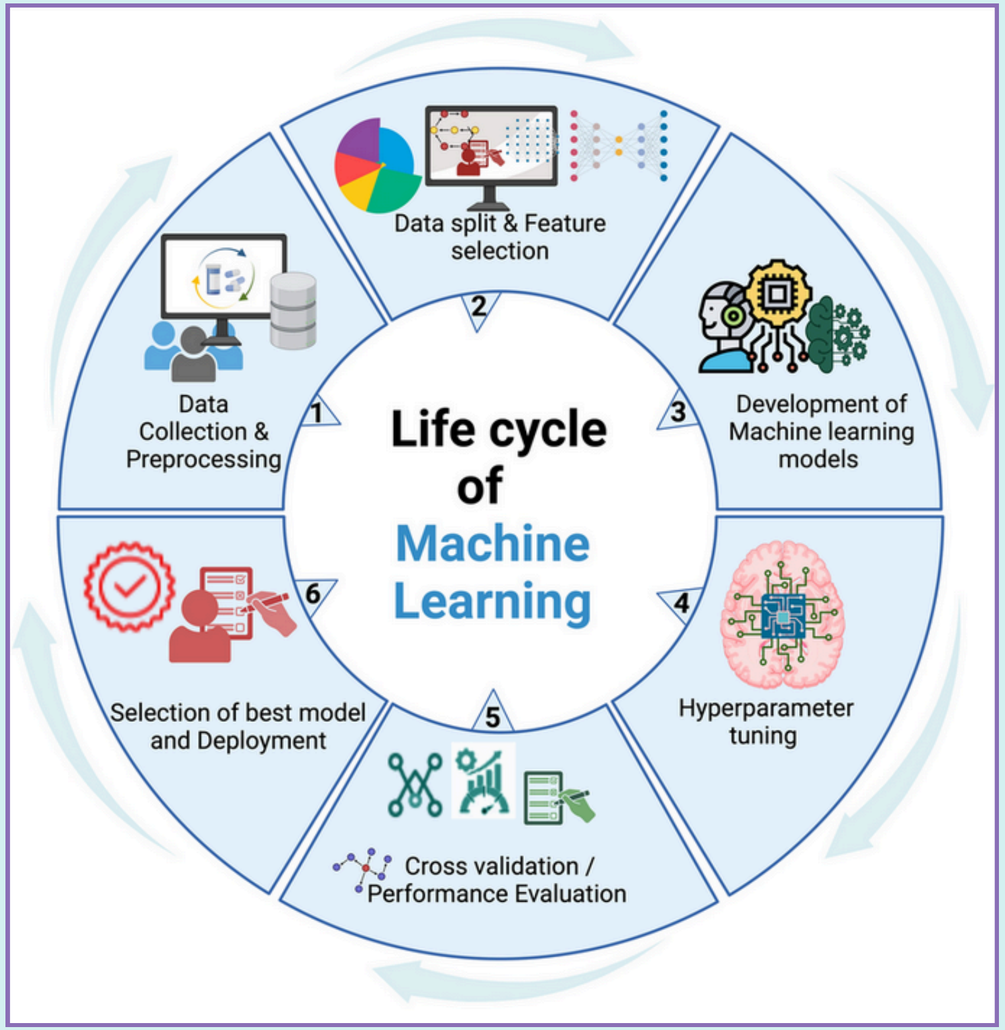
To use any model in scikit-learn, our data must be in a very specific format:

X is a (n,p) numpy array n input observations in dimension p.
y is a (n,p_out) numpy array expected outputs.



Developing your ml model





Common pre-processing techniques

1. **Handling missing values -**
   - Replace with mean / median (numerical),
   - Replace with most frequent value (categorical)
2. **Feature Scaling -** Feature scaling changes the range of numerical values so that all features are comparable (Standardization, Normalization)
3. **Feature Encoding-** Encoding converts categorical data into numerical form
   - One-hot encoding → converts categories into binary columns
   - Label encoding → converts categories into numbers
4. **Avoiding Data Leakage -** Fit preprocessing steps only on training data. Apply the same transformation to test data
5. **Feature Selection**
6. **Feature Engineering -** Better features often matter more than better models
   - Eg. PCA reduces the number of features while preserving most of the important information. (**Dimensionality Reduction**)

Transformers

- Data is typically formatted as a **NumPy array** where **each row represents a data sample and each column represents a feature**. Target values (labels) are stored separately.

- **scikit-learn transformers** provide a clean and systematic way to **preprocess data.**

- Transformers **prepare raw data into a suitable numerical form** so that machine learning models can learn effectively.

1. First it learns parameters from data → **fit**
2. Then it transforms data → **transform**

- **Why do we need this?** Machine learning **models are sensitive to how data is represented**. If features are on very different scales or formats, models may perform poorly.

Example

```python
# (2D array is required by sklearn; here there is only 1 categorical feature and 3 samples)
colors = np.array([["Red"], ["Blue"], ["Green"]])

# sparse_output=False: Returns a simple numpy array instead of a sparse matrix (easier to read)
encoder = OneHotEncoder(sparse_output=False)

# 'fit' learns the categories (Blue, Green, Red)
# 'transform' converts them to binary vectors
colors_encoded = encoder.fit_transform(colors)

print("Original Data:\n", colors)

print("\nOne-Hot Encoded Matrix:\n", colors_encoded)
# Note: Categories are sorted alphabetically by defaul
# Index 0 = Blue, Index 1 = Green, Index 2 = Red
# So, Red -> [0. 0. 1.] (3rd alphabetical category)

print("\nLearned Categories:", encoder.categories_)

# Inverse Transform
original_restored = encoder.inverse_transform(colors_encoded)
print("\nRestored Data:\n", original_restored)
```

```
Original Data:
 [['Red']
 ['Blue']
 ['Green']]

One-Hot Encoded Matrix:
 [[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]

Learned Categories: [array(['Blue', 'Green', 'Red'], dtype='<U5')]
```

Please find code in the shared notebook

Example

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> scaler.fit(data)
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```


also refer to “Scaling” section in shared notebook

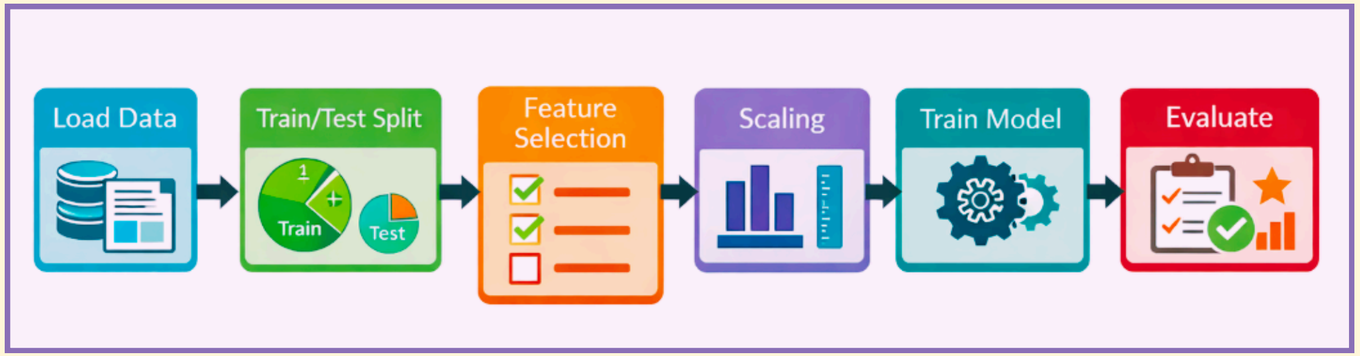
California Housing Dataset

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80
17848	6.6227	20.0	6.282147	1.008739	2695.0	3.364544	37.42	-121.86
6252	2.5192	28.0	4.345361	1.074742	1355.0	3.492268	34.04	-117.97
9389	7.9892	37.0	6.052758	0.954436	999.0	2.395683	37.91	-122.53
6113	1.5000	5.0	3.620579	1.016077	819.0	2.633441	34.13	-117.90
6061	6.4266	5.0	6.730284	1.030609	9427.0	3.394671	34.02	-117.79

```
1.03
3.821
1.726
0.934
0.965
2.648
1.573
5.00001
1.398
3.156
```

The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars($100,000)

Overview



Takes raw California housing data and predicts house prices

```
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

selector = SelectKBest(f_regression, k=5)
X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

model = LinearRegression()
model.fit(X_train_selected, y_train)

y_pred = model.predict(X_test_selected)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)

selected_features = X.columns[selector.get_support()]
print("Selected Features:", selected_features)
```

- Example for splitting dataset into train and test :

```
X_train, X_test, y_train, y_test = train_test_split(X_house, y_house, test_size=0.2, random_state=42)
```

- Keep top 5 features (feature selection):

```
selector = SelectKBest(f_regression, k=5)
X_train_fs = selector.fit_transform(X_train, y_train)
X_test_fs = selector.transform(X_test)
```

- Scale features to a standard range
- Fit model on training data

```
model = LinearRegression()
model.fit(X_train_final, y_train)

y_pred = model.predict(X_test_final)
```

- Evaluation

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

Cross Validation

We have been splitting data once into training and testing sets. But a single split may give a misleading estimate of model performance.

In k-fold cross-validation, the dataset is divided into k parts. Each part is used once as a test set while the remaining parts are used for training.

Cross-validation gives a more reliable estimate of model performance than a single train–test split.

```
from sklearn.model_selection import cross_val_score

# Instead of one split, we split the data 5 different ways (cv=5) for more reliable evaluation
scores = cross_val_score(pipeline, X_house, y_house, cv=5, scoring='r2')

print("Cross-Validation Scores:", scores)
print("Average Accuracy:", scores.mean())
```

Pipelines

In real machine learning workflows, we don’t just train a model - we preprocess data, select features, and then train the model.

scikit-learn pipelines allow us to bundle all these steps together.
A pipeline chains preprocessing steps and a model into a single object.

During training, each transformer is fit only on training data, and the same transformations are applied to test data automatically

Pipelines allow us to treat preprocessing and modeling as a single, safe, reusable unit.

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),      # Step 1: Scale
    ('selector', SelectKBest(k=5)),    # Step 2: Select Features
    ('regressor', LinearRegression())  # Step 3: Model
])

# Now we can treat the whole pipeline as a single model
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

Thank You!